

Testing Brain-Inspired Mechanisms for Task-Incremental Continual Reinforcement Learning: A Controlled Ablation Study

Anonymous authors

Paper under double-blind review

Abstract

Brain-inspired mechanisms such as spiking neurons, local plasticity, homeostasis and sleep-like consolidation are proposed as remedies for catastrophic forgetting in continual reinforcement learning (RL), usually several at once. We ask which of them help when each is isolated and compared with a matched non-biological control. In task-incremental MiniGrid sequences (the task identity is given) with small PPO agents, we test six mechanisms one at a time, with tests prospectively specified in a version-controlled experiment log, 3–10 seeds per arm, 95% confidence intervals and matched budgets. A seventh, Transformer working memory, is inconclusive: no benchmark we tried was shown to be valid (requiring memory and learnable within budget). No mechanism produced a statistically reliable improvement in final accuracy or forgetting beyond its matched control; given the seed counts, these results rule out large effects rather than show equivalence, and they do so only where the intervals are narrow. On DoorKey-6x6, the tested stabilised reward-modulated STDP rule was not distinguishable from a random update of the same size (AUC difference $+0.033$ $[-0.260, +0.326]$, an interval too wide to exclude large effects). Sleep-like consolidation removed most of naive fine-tuning’s forgetting but, at an exactly matched replay budget, was not distinguishable from experience replay (final-accuracy difference $+0.017$ $[-0.026, +0.060]$). Two narrower effects stand. Homeostasis sped up learning in a spiking encoder trained with backprop-free encoder credit assignment (direct feedback alignment; the heads use exact gradients), with an AUC difference of $+0.326$ $[+0.101, +0.551]$ (Holm $p = 0.017$), without making it reliable. With a small replay buffer, a shared trunk with per-task heads and sleep reached similar mean accuracy to full-width isolation (one network per task; 0.838 vs 0.833; 3 seeds, wide CI) in less than half the total memory; at the default buffer, isolation was more memory-efficient. A post-freeze follow-up qualifies this: on a 5-task CNN sequence with 10 fresh seeds, isolation with quarter-width networks beat a shared trunk trained with LwF or with replay at equal or lower memory (0.972 vs 0.944 and 0.920), so the comparison depends on isolation’s network width. A spiking encoder works at fewer operations per frame than a sparsified CNN, but its measured training cost in our CPU implementation is $5.2\times$ higher, and its energy advantage rests on an estimated arithmetic energy proxy. Seven earlier positive findings shrank or disappeared under controls; we report each with its cause.

1 Introduction

A continual reinforcement learning (RL) agent learns a sequence of tasks while its parameters keep changing. With plain gradient-based fine-tuning, learning a new task overwrites what earlier tasks needed: catastrophic forgetting. The problem is framed as a stability–plasticity trade-off [LIT] (Khetarpal et al., 2022; Kirkpatrick et al., 2017), and the standard remedies are regularisation towards old parameters (EWC; Kirkpatrick et al., 2017), replaying stored experience (CLEAR; Rolnick et al., 2019) and isolating parameters per task (e.g. PackNet; Mallya & Lazebnik, 2018).

A second family of remedies is inspired by the brain, and it is popular. Spiking neural networks (SNNs) promise event-driven, low-energy computation [LIT] (Fang et al., 2023; Zhou et al., 2026). Local learning rules such as STDP and its reward-modulated three-factor variants are proposed as biologically plausible complements or alternatives to backpropagation [LIT] (Lee et al., 2025; Huo et al., 2025). Sleep-like offline replay with local plasticity has been reported to recover forgotten tasks [LIT] (Tadros et al., 2022), and homeostatic plasticity keeps biological firing rates in a working range [LIT] (Turrigiano & Nelson, 2004). These mechanisms are often introduced together, in one system, and compared against an unmodified baseline. When such a system does better, it is hard to know which mechanism did the work, or whether a simpler non-biological control would have done the same.

We ask a narrower question: *in a controlled task-incremental continual RL setting, which brain-inspired mechanisms improve learning or retention when each is isolated and compared with a matched control?* We build one agent (an SNN or CNN encoder, optional Transformer working memory, PPO) and switch six mechanisms on and off: the spiking encoder, STDP, homeostasis, direct feedback alignment (DFA) as backprop-free encoder credit assignment (the heads use exact gradients), sleep-like consolidation, and a shared trunk with per-task heads versus parameter isolation. Each gets a control chosen to remove the most obvious alternative explanation: a same-size random update for STDP, a replay baseline with an exactly matched number of replayed samples for sleep, and total-memory accounting for the shared trunk. A seventh mechanism, Transformer working memory, was to be compared with a memoryless baseline and a frame stack, but that comparison is inconclusive because benchmark validity was not established.

Headline answer. [OURS] No mechanism produced a statistically reliable improvement in final accuracy or forgetting beyond its matched non-biological control. Given the seed counts, these results rule out large effects rather than show equivalence, and they rule out large effects only where the intervals are narrow (Section 3.5). Two narrower effects stand, and one of them is conditional. Homeostasis speeds up learning when the spiking encoder uses backprop-free encoder credit assignment (DFA; the heads use exact gradients), without making it reliable. With a small replay buffer, a shared trunk with per-task heads and sleep reaches similar mean accuracy to full-width isolation (0.838 vs 0.833; 3 seeds, wide CI) in less than half the memory; at the default buffer size, isolation is more memory-efficient; and a post-freeze follow-up found that isolation with quarter-width networks beats shared-trunk methods at equal or lower memory on a 5-task CNN sequence, so this exception holds only against full-width isolation. The spiking encoder reaches a lower operation-count regime than a sparsified CNN, at a small accuracy cost; whether that saves energy depends on neuromorphic hardware we did not use. The Transformer question is inconclusive.

Contributions.

1. A controlled ablation of six brain-inspired mechanisms for task-incremental continual RL, each switched on alone and compared with a control chosen to rule out its most obvious alternative explanation, plus a Transformer working-memory comparison that remained inconclusive because no benchmark both required memory and was learnable within budget (Sections 4 and 5).
2. Budget-matched comparisons that change the conclusion of the unmatched ones: sleep versus replay at an exactly matched number of replayed samples; a shared trunk with per-task heads versus isolation per unit of total memory (parameters plus replay buffer) rather than per parameter; and, after the freeze, isolation at a smaller network width (Sections 5.4 and 5.5).
3. Tests prospectively specified in a version-controlled experiment log, with effect sizes, 95% confidence intervals and multiple-comparison correction, every result labelled as confirmatory, exploratory or descriptive (Section 3.5), and a record of seven earlier positive findings that shrank or disappeared under controls, each with its cause (Section 6).
4. Code, frozen results, a script that regenerates every number in this paper from the result files, and an offline results page with the trained agents (Reproducibility statement).

How to read the claims. We label statements as literature claims [LIT], our interpretations [INTERP], or results of our experiments. Each result of ours carries one of three labels. [CONFIRMATORY]: a test whose

comparison, statistic, seed count and stopping rule were prospectively specified in the version-controlled experiment log before its runs, and that was run as specified. [EXPLORATORY]: a comparison that was not specified in advance (the first two phases of the project, pilots, context arms, analyses chosen after seeing the data); its p -values are uncorrected and should be read as descriptive. [DESCRIPTIVE]: numbers reported without a test (means, ratios, operation counts, measured CPU time, memory accounting). [POST-FREEZE] marks the one result obtained after the main results were frozen (Section 5.5). Summaries that combine several results are marked [OURS] and refer to the labelled results. Literature numbers were taken from the cited papers and are marked as such.

2 Related work

Continual RL. [LIT] Khetarpal et al. (2022) review continual RL and its desiderata: retention, forward transfer, and ideally operation without task identities or boundaries. EWC (Kirkpatrick et al., 2017) penalises moving parameters that were important for earlier tasks (Fisher-weighted); it protects old tasks at the cost of plasticity for new ones. Learning without Forgetting (LwF; Li & Hoiem, 2018) instead regularises the network’s *outputs*: it distils a copy of the network taken before the new task on new-task inputs, so it needs no stored data. CLEAR (Rolnick et al., 2019) mixes new experience (for plasticity) with replayed old experience (for stability), trains on both with the off-policy V-trace actor-critic loss, and adds behavioural cloning on the replayed experience: a KL term towards the stored policy and an L2 term towards the stored value. It needs no task identities, and it is a strong, simple continual RL method. Our replay baseline and our sleep phase both reuse CLEAR’s cloning losses; Section 4.5 states exactly how they differ from CLEAR and from each other. Continual World (Wolczyk et al., 2021) argues that forgetting alone is an incomplete metric and that forward transfer must be measured; we report their forward-transfer measure. PackNet (Mallya & Lazebnik, 2018) isolates parameters per task by pruning and freezing, which removes forgetting by construction; our isolation baseline is the simpler one-network-per-task variant. We measure forgetting as in Chaudhry et al. (2018).

Sleep and consolidation. [LIT] Tadros et al. (2022) implement a sleep-like phase in feed-forward networks (offline, noisy input, local unsupervised Hebbian plasticity) and report that it recovers previously forgotten tasks in incremental classification. Our sleep phase is *inspired* by this line of work but is different: it is offline replay plus distillation in an RL setting, optionally with STDP (Section 4.5), and it relies on a replay buffer. Yu et al. (2025) distil accumulated experience of LLM agents into parameters instead of retrieving raw trajectories, which motivates counting the memory that replay needs (Section 5.5).

Spiking networks for RL and control. [LIT] SNNs are trained end to end with surrogate gradients (Neftci et al., 2019). Van den Berghe et al. (2025) study the surrogate-gradient slope and show that it matters much more in RL than in supervised learning; they also describe a warm-up problem for stateful SNNs in RL. Our encoder resets its membrane at every environment step, so it avoids that problem, and it has no memory across frames (Section 4.1). Lee et al. (2025) and Huo et al. (2025) review biologically plausible learning rules for SNNs, including STDP and reward-modulated three-factor rules (Bi & Poo, 1998; Frémaux & Gerstner, 2016). The Spiking Decision Transformer (Pandey & Biswas, 2025) combines spiking attention with three-factor plasticity for offline RL on classic control tasks; Spikingformer (Zhou et al., 2026) points out that many “spiking” architectures hide non-spike floating-point computation, a warning we take into account when counting operations. SpikingJelly (Fang et al., 2023) and snnTorch (Eshraghian et al., 2023) are common SNN toolkits; we use our own LIF implementation, checked against snnTorch.

Local and weight-transport-free learning. [LIT] Direct feedback alignment (Nøkland, 2016) projects the output error to hidden layers through fixed random matrices, avoiding the weight transport that back-propagation requires. Homeostatic plasticity (Turrigiano & Nelson, 2004) adjusts excitability to keep firing rates near a set point.

Estimated arithmetic energy proxies. [LIT] A common proxy counts synaptic operations and charges an accumulate (about 0.9 pJ) per spike-driven operation and a multiply-accumulate (about 4.6 pJ) per dense

operation, using 45 nm estimates (Horowitz, 2014). Such numbers are estimates of arithmetic energy, not measured power, and they assume event-driven hardware.

3 Setup

3.1 Environments and task sequences

All environments are MiniGrid (Chevalier-Boisvert et al., 2023) gridworlds with the standard 7×7 egocentric symbolic view, which we encode as 20 one-hot channels (object type, colour and state).

fetch3 and fetch5 (continual). A room (6×6 interior) contains three (fetch3) or five (fetch5) distinct objects at random positions. Task k rewards picking up object k (reward $1 - 0.9t/T_{\max}$, the MiniGrid convention); picking up any other object ends the episode with reward 0. Observations are statistically identical across tasks, so the same state demands different actions in different tasks: the setting guarantees interference under naive fine-tuning, while navigation and pick-up are shared skills that allow forward transfer. The agent is given the task identity (a task embedding and one actor-critic head per task). This is the standard *task-incremental* setting; without the task identity the tasks are not merely harder but unsolvable, since the goal is not visible (Section 7). Tasks are trained in order for a fixed number of frames each (150k per task for the CNN and SNN agents, 400k for the SNN+Transformer agent).

DoorKey-6x6 (single task). Used for the local-plasticity questions: the agent must pick up a key, open a door and reach the goal; 400k frames. It is hard enough that the encoder’s learning matters (a frozen random encoder learns more slowly), and outcomes are bimodal (a run either solves it or stays near zero).

MiniGrid-Memory (S7, S11, S13). The agent sees a cue object at one end of a corridor and must choose the matching object at the other end. We use these maps for the Transformer question (Section 5.6).

3.2 Agent and training

The encoder is either a CNN (three 2×2 convolutions with 16, 32 and 64 channels and a linear layer to 128 features) or an SNN with the same topology, leaky integrate-and-fire (LIF) neurons, 4 timesteps per environment step, a learnable leak (initial $\beta = 0.9$) and a surrogate gradient (slope 25) (Neftci et al., 2019). Actor and critic heads are small MLPs. Training is PPO (Schulman et al., 2017) in a single-file CleanRL-style implementation (Huang et al., 2022): 16 parallel environments, 128 steps per rollout, 4 epochs, minibatches of 256, clip 0.2, $\gamma = 0.99$, GAE $\lambda = 0.95$, early stopping at an approximate KL of 0.03, Adam with learning rate 0.001 (0.0003 for the SNN+Transformer agent). Everything runs on CPU (one thread per run).

Shared trunk with per-task heads. Every non-isolation agent is a shared trunk with per-task heads. The trunk (the encoder, plus the Transformer in the SNN+Transformer agent) is shared by all tasks; each task adds one actor-critic head and one task-embedding vector. For the CNN agent the trunk has 142,832 parameters and each task adds 17,160 (a 17,032-parameter head and a 128-dimensional embedding), so the 3-task agent has 194,312 parameters; the SNN and SNN+Transformer agents differ only in the trunk (the shared SNN+Transformer agent has 327,800). The parameters that scale with the number of tasks are therefore the heads and embeddings, and, for replay and sleep, the replay buffer (5,000 stored states per task by default). Isolation instead adds a full network of 159,864 parameters (CNN) per task.

3.3 Baselines: three strategies against forgetting

The non-biological comparators follow the three standard strategies [LIT] (Khetarpal et al., 2022). *Regularisation*: EWC (Kirkpatrick et al., 2017) penalises changes to parameters that were important for earlier tasks; our online variant keeps one accumulated Fisher estimate and one anchor copy of the parameters, so its storage does not grow with the number of tasks ($\lambda = 10, 100, 1000$). *Replay*: CLEAR-style replay (Rolnick et al., 2019) interleaves cloning losses on stored states with learning the new task (Section 4.5); it stores a buffer that grows with the number of tasks. *Capacity isolation*: one fresh network per task, which

cannot forget by construction and whose parameters grow by a full network per task. Naive fine-tuning of the shared trunk is the lower bound. Sleep-like consolidation is compared with replay because it uses the same buffer and losses. The post-freeze follow-up (Section 5.5) adds Learning without Forgetting (LwF; Li & Hoiem, 2018), a functional regulariser: at the start of each task it keeps a frozen copy of the network (stored in int8, 0.23 MB for the 5-task CNN agent) and, on every PPO minibatch, distils its policy and value on 128 current-task states, each paired with a random earlier task’s head. It stores no data between tasks.

3.4 Metrics

$R_{i,j}$ is the mean evaluation return on task j after training on task i (100 episodes, actions sampled from the policy). For T tasks: $\text{ACC} = \frac{1}{T} \sum_j R_{T,j}$ (final average performance); $\text{BWT} = \frac{1}{T-1} \sum_{j < T} (R_{T,j} - R_{j,j})$; $\text{FORGET} = \frac{1}{T-1} \sum_{j < T} (\max_{i < T} R_{i,j} - R_{T,j})$ (Chaudhry et al., 2018); forward transfer FWT is the normalised gain in area under the training curve on tasks 2.. T relative to a from-scratch network on the same task and seed (Wołczyk et al., 2021). For single-task runs we report the solve rate (final evaluation return above 0.5 over 200 episodes) and the AUC (mean training return over the run, a sample-efficiency measure).

Memory accounting. For the shared-trunk question we count total memory as fp32 parameters plus the replay buffer at its measured storage cost: 188 bytes per stored state for the CNN and SNN agents (observation, mask, task id, teacher logits and value) and 632 bytes for the SNN+Transformer agent (4-frame windows), times 5,000 states per task by default.

3.5 Statistics

Each arm has 3–10 seeds. For solve rates we use Fisher’s exact test and the Newcombe (Wilson score) interval for the difference of proportions (Newcombe, 1998); for continuous metrics, Welch’s t -test (Welch, 1947) and the Welch 95% confidence interval of the difference of means. Where a prospectively specified family contains several contrasts we apply Holm’s correction (Holm, 1979). With so few seeds, CIs are wide; we treat them as the primary result. A non-significant difference is read as “no large effect” only where its interval excludes large effects, and never as equivalence.

Seed sets. Within every comparison, all arms use the same seed set: seeds 1 to n for the frozen runs and 201–210 for the post-freeze follow-up, so each seed appears in every arm of the comparison. The one exception is the unmatched SNN+Transformer sleep arm (seeds 1–6), which Table 3 and Figure 3 show next to replay on seeds 1–8; the budget-matched test (Section 5.4) uses seeds 1–8 in both arms, and the seed-count history in Table 4 restricts both arms to the same seeds. The tests treat the arms as independent samples.

Prospective specification. The work was done in four phases. The first two were exploratory. For the last two, the comparisons, test statistics, seed counts and stopping rules were recorded in the project’s version-controlled experiment log before the runs, one planned test per question; the post-freeze follow-up was specified in the same way in its own log (`explore/EXPLORE.md`) before its runs. This is a project record, not a public registry. We call these tests *prospectively specified in a version-controlled experiment log* (prospectively specified, for short) and label their results [CONFIRMATORY]. Results not specified in advance are labelled [EXPLORATORY] and enter no confirmatory test.

4 Mechanisms and their controls

Table 1 lists each mechanism, how it is implemented and what it is compared against.

4.1 Spiking encoder

The SNN integrates each frame over 4 internal timesteps and its membrane state is reset at every environment step, so it has no memory across frames; we verified this empirically (identical output for identical frames

Table 1: Mechanisms, implementations and controls. Each row is switched on and off independently. The first six rows were tested; the Transformer comparison is inconclusive because benchmark validity was not established.

Mechanism	Implementation	Control / comparator
Spiking encoder	LIF CNN, 4 timesteps per frame, surrogate gradients; membrane reset every environment step	Same-topology CNN; op count at matched accuracy
STDP	Reward-modulated three-factor STDP on encoder synapses, added to the backprop update: $\Delta w = \alpha \Delta w_{\text{STDP}} + \Delta w_{\text{grad}}$, stabilised by mean-centring and homeostasis	Same-RMS random update + homeostasis; homeostasis alone
Homeostasis	Per-neuron bias pushes firing rate towards 0.15	Same agent without it
DFA	Backprop-free encoder credit assignment: encoder layers learn from the output error through fixed random feedback (heads use exact gradients)	DFA without homeostasis; backprop
Sleep	Offline phases: old-task distillation on stored states + current-task self-distillation	CLEAR-style replay (cloning losses, interleaved), replayed samples matched exactly; EWC; naive
Shared trunk	One trunk shared by all tasks, per-task heads and task embeddings	One network per task (isolation), at equal total memory; after the freeze, isolation at quarter width
Transformer memory (inconclusive)	Transformer over the last K encoded frames (past frames only)	Memoryless agent; frame stack of K frames

regardless of history). We count event-driven operations per frame: synaptic operations for the SNN (non-zero presynaptic spikes times fan-out, summed over timesteps) and non-zero-activation multiply-accumulates for a CNN whose activations are sparsified with an L1 penalty. The estimated arithmetic energy proxy multiplies these counts by the literature constants above. To compare accuracy against operations without RL noise, the op-count frontier is measured by behaviour cloning of a trained DoorKey-6x6 teacher: both encoders plus an identical linear readout are trained with an activity penalty swept over a range, and we report held-out action accuracy and the closed-loop return of the cloned policy (2 seeds per point).

4.2 STDP and the random-update control

After every PPO optimiser step, spikes recorded in the minibatch’s forward pass produce a local STDP update for each encoder layer (eligibility traces, pre/post pairing), multiplied by a third factor, the normalised PPO advantage of the sample (reward-modulated STDP) (Frémaux & Gerstner, 2016; Lee et al., 2025). The update is normalised to unit RMS and scaled by α times the learning rate, so $\alpha = 1$ moves each weight about as much as one Adam step. The *stabilised* version mean-centres pre- and post-synaptic spikes (a covariance rule, no drift under independent firing) and adds homeostasis. The *random* control applies an update with the same per-layer RMS but a random direction, also with homeostasis, so it separates “STDP helps” from “any perturbation of this size helps”. We use $\alpha = 0.03$ for the prospectively specified comparison.

4.3 Homeostasis

Each spiking neuron’s bias is nudged towards a target firing rate: $b \leftarrow b + \eta(0.15 - r)$ with $\eta = 0.01$ and r the neuron’s rate in the minibatch.

4.4 Direct feedback alignment

In the DFA agents, credit assignment in the encoder is backprop-free: the encoder’s layers receive the output error through fixed random feedback matrices instead of backpropagated gradients (Nøkland, 2016), while the actor and critic heads use exact gradients. This gives the encoder a global learning signal without weight transport.

4.5 Sleep-like consolidation and the replay baseline

Every 25 PPO updates, and once at the end of each task, the agent goes offline for 700 gradient steps on two losses: (i) distillation towards the stored policy and value on states saved at the end of each earlier task (KL divergence plus value MSE; the same buffer the replay baseline uses), and (ii) self-distillation of the current task towards a frozen snapshot of the network taken at sleep onset, on a reservoir of recent wake states. Optionally, local STDP runs on the spikes produced during sleep. The *replay* baseline adds the same distillation loss on one replayed minibatch to every PPO minibatch while learning later tasks, as in CLEAR.

Relation to CLEAR. Both methods reuse CLEAR’s behavioural-cloning losses (Rolnick et al., 2019), and they differ from CLEAR, and from each other, in specific ways. (i) Our *replay* baseline keeps CLEAR’s cloning terms (policy KL and value regression on replayed states) but not its off-policy RL loss on replayed experience: PPO is on-policy, and replayed transitions cannot enter its objective without an importance-weighting scheme such as V-trace, which we did not add. The cloning targets are the policy and value of the network at the end of the task a state comes from, stored with the state, rather than those of the policy that generated the experience. As in CLEAR, replay is interleaved with every update on the new task. (ii) Our *sleep* phase uses the same buffer and the same cloning losses, but applies them offline, in scheduled phases without environment interaction, and adds self-distillation on recent current-task states. With the number of replayed states matched, the sleep–replay comparison therefore isolates *when* replay happens, plus the self-distillation term; it does not compare sleep with the full CLEAR method. (iii) Unlike CLEAR, both use the task identity (per-task heads), because our setting is task-incremental.

Replay budget. The two methods consume different numbers of replayed states. Replay’s count depends on how many PPO minibatches run, which varies by seed because of KL early stopping (0.91M–1.48M per SNN+Transformer run); sleep’s is fixed by its schedule (1.43M). For the prospectively specified comparison, each sleep run was given a total replay budget equal to the measured count of the same-seed replay run, spread evenly over its sleep phases (*sleep, replay-matched*); the match was exact for every seed.

4.6 Shared trunk with per-task heads versus isolation

The shared agent keeps one trunk and adds a head and a task embedding per task (Section 3.2). Isolation trains a fresh network per task: zero forgetting by construction and a full network’s parameters per task. We compare them per parameter, as is common, and per unit of total memory, which includes the replay buffer that the shared-trunk methods need. In the frozen study, each isolated network has the same width as the shared trunk; the post-freeze follow-up also tests a narrower isolated network.

4.7 Transformer working memory

The SNN+Transformer agent encodes each of the last K frames and runs a one-layer Transformer encoder (Vaswani et al., 2017) over them, reading out the newest position; the window holds only past frames, and padding before the episode start is masked. On the fetch tasks $K = 4$. The non-attention baselines are a frame stack (the last K frames as input channels) and, implemented but not run, a GRU over the same encoded frames.

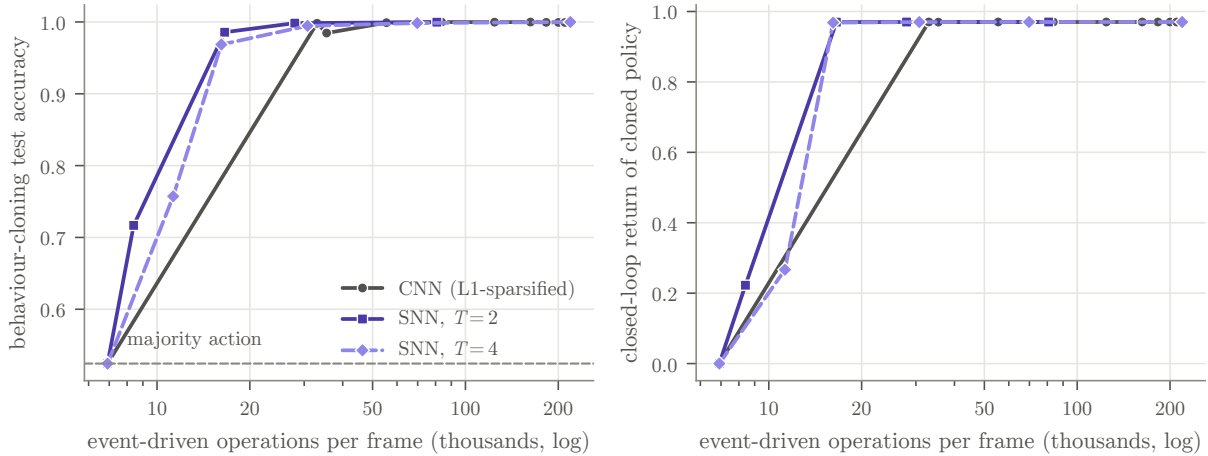


Figure 1: Accuracy against operation count for the spiking and the conventional encoder (descriptive). Each encoder, with an identical linear readout, is trained by behaviour cloning of a trained DoorKey-6x6 PPO teacher with an activity penalty; each point is one penalty value (mean of 2 seeds, the same seeds at every point). The x -axis counts event-driven operations per frame (synaptic operations for the SNN, non-zero multiply-accumulates for the L1-sparsified CNN; log scale). Left: held-out action accuracy (dashed: majority-action rate). Right: closed-loop return of the cloned policy. The SNN keeps working below the lowest operation count at which the CNN still works.

5 Results

5.1 Spiking encoder: a lower operation-count regime, not more accuracy

[DESCRIPTIVE] Where both encoders work, they are equally accurate at matched operation counts (Figure 1). The unpenalised CNN uses 210k event-driven operations per frame (accuracy 0.999). The L1-sparsified CNN still works at 33k operations (accuracy 0.998), but a stronger penalty silences the whole network at once (6.9k operations, accuracy 0.524, the majority-action rate). The SNN keeps working below that floor: with 4 timesteps reduced to 2 it reaches 16.6k operations at accuracy 0.986 and a closed-loop return of 0.970, about $2.0\times$ fewer operations than the CNN floor; $T = 4$ behaves similarly (16.2k, 0.969). Under the estimated arithmetic energy proxy (literature constants for 45 nm arithmetic), this point costs 14.9 nJ per frame against 126 nJ for the sparsest working CNN and 1653 nJ for the dense CNN.

[DESCRIPTIVE] Measured cost in our implementation: on the CPU we actually used, one PPO training step takes 80 ms for the SNN against 16 ms for the CNN ($5.2\times$), because the SNN simulates 4 timesteps. The unit is CPU process time on one thread for one forward pass, backward pass and Adam update on a minibatch of 256 DoorKey-6x6 observations (single-task networks), averaged over 5 repetitions after one warm-up step (`scripts/benchmark_compute.py`). [INTERP] The SNN’s advantage is access to a lower-operation regime, not better accuracy, and the energy numbers assume event-driven hardware. Other CNN sparsifiers (e.g. k -winner-take-all) might move the CNN floor; we did not test them.

Robustness per unit of compute. [DESCRIPTIVE] Dividing final continual accuracy (fetch3) by training compute (the measured per-step CPU cost above times frames), the plain CNN scores 0.978 per unit, $5.7\times$ the best spiking variant (SNN with sleep, 0.173); SNN with homeostasis scores 0.164, DFA with homeostasis 0.090, stabilised STDP 0.084 and the SNN+Transformer agent 0.056. Encoders trained by local STDP only never solved DoorKey (0/6 runs).

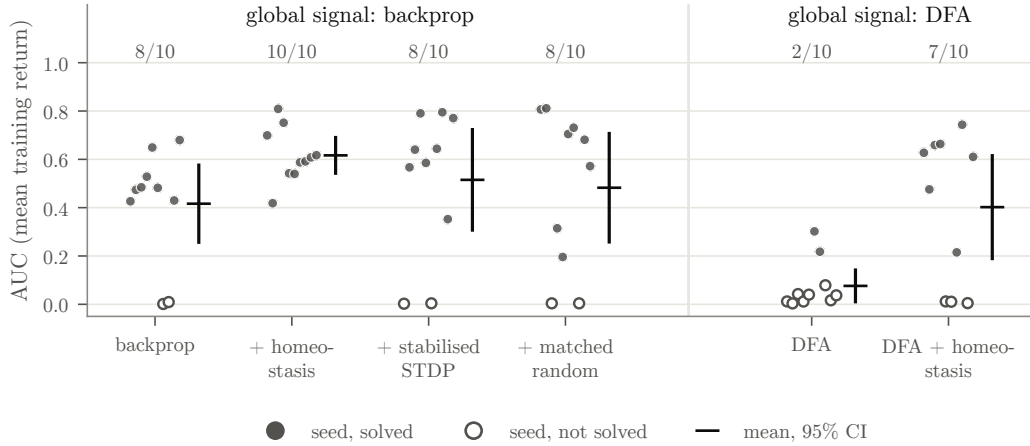


Figure 2: Local-plasticity arms on DoorKey-6x6 (confirmatory: prospectively specified; 400k frames, 10 seeds per arm, seeds 1–10 in every arm). Each dot is one seed’s AUC (mean training return over the run); filled dots solved the task (final evaluation return above 0.5), open dots did not. Black bars: mean with 95% CI. Numbers above the dots: seeds solved. Left group: backpropagation as the global learning signal, alone or with homeostasis, stabilised reward-modulated STDP, or a random update of the same size. Right group: backprop-free encoder credit assignment by direct feedback alignment (DFA; heads use exact gradients), with and without homeostasis.

Table 2: Prospectively specified local-plasticity contrasts on DoorKey-6x6 (confirmatory; 10 seeds per arm, seeds 1–10 in every arm). Differences are the first arm minus the second, with 95% CIs (Newcombe for solve rates, Welch for AUC). Holm-corrected p across the four contrasts, separately per statistic.

Contrast	Δ solve rate [95% CI]	Fisher p (Holm)	Δ AUC [95% CI]	Welch p (Holm)
STDP vs homeostasis alone	−0.20 [−0.51, +0.11]	0.474 (1.000)	−0.101 [−0.323, +0.121]	0.338 (0.724)
STDP vs matched random	+0.00 [−0.34, +0.34]	1.000 (1.000)	+0.033 [−0.260, +0.326]	0.818 (0.818)
homeostasis vs backprop	+0.20 [−0.11, +0.51]	0.474 (1.000)	+0.200 [+0.024, +0.377]	0.029 (0.117)
random vs homeostasis	−0.20 [−0.51, +0.11]	0.474 (1.000)	−0.134 [−0.372, +0.104]	0.241 (0.724)

5.2 STDP: the tested rule was not distinguishable from a same-size random update

[CONFIRMATORY] With backpropagation as the global signal, the four arms solve DoorKey-6x6 at similar rates: backprop 8/10, backprop with homeostasis 10/10, with stabilised STDP 8/10, with the matched random update 8/10 (Figure 2). All four prospectively specified contrasts are null after Holm correction (Table 2). The tested rule (stabilised reward-modulated STDP at $\alpha = 0.03$, added to the backprop update) was not distinguishable from a random update of the same size (AUC difference +0.033 [−0.260, +0.326], $p = 0.818$), and its point estimate was slightly below homeostasis alone. These intervals are wide: with 10 seeds per arm they do not exclude a large effect in either direction, so this is “no detectable effect of this rule in this setup”, not evidence that STDP cannot help.

[EXPLORATORY] Unstabilised STDP hurt. In an earlier DoorKey-5x5 experiment, reward-modulated STDP with $\alpha = 1$ solved 0/3 runs against 3/3 for backprop (AUC difference −0.502 [−0.834, −0.170], $p = 0.015$), with a mean spike rate of 0.39 over the last updates against 0.20 for backprop. A larger stabilised α (0.3) was also worse (2/3 solved, context arm). [INTERP] The frame is static across the SNN’s timesteps, so the same pre-post pairings repeat and a mostly potentiating rule feeds back on itself (weights up, rates up, more correlated spikes). Stabilisation removes the runaway but leaves nothing measurable beyond a random perturbation.

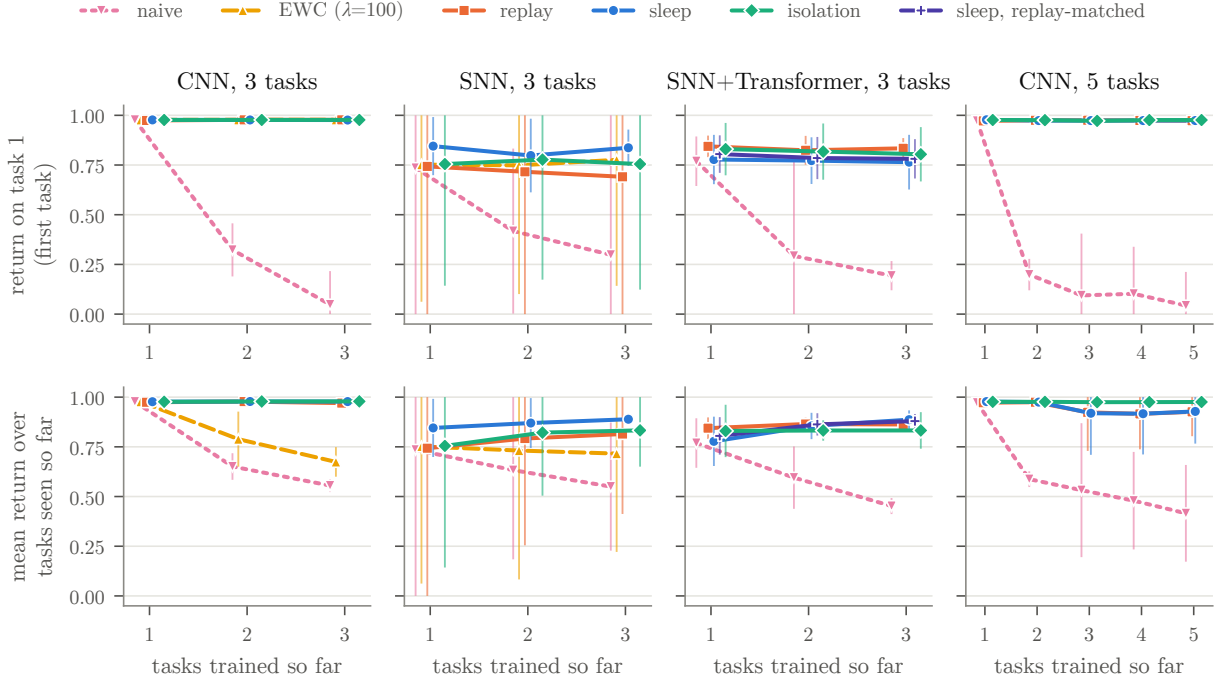


Figure 3: Forgetting on the continual task sequences, by method (exploratory, except the replay-matched sleep arm, which belongs to the confirmatory test of Figure 4). Columns: agent and sequence. Top row: return on the first task after training on each task in turn (a flat line means task 1 is not forgotten). Bottom row: mean return over the tasks seen so far. Mean over seeds with 95% CIs (3 seeds per arm, seeds 1–3, except the SNN+Transformer agent: replay seeds 1–8, replay-matched sleep 1–8, sleep 1–6); the CIs are clipped to $[0, 1]$, the range of returns, and with 3 seeds the SNN intervals span most of it. Methods are offset horizontally so that overlapping lines stay visible. Colours mean the same method in every figure.

[EXPLORATORY] Inside sleep, the same pattern holds on the SNN trunk (fetch3): vanilla STDP during sleep collapsed performance (ACC 0.163, 1 seed), while stabilised STDP (0.881 ± 0.028) was indistinguishable from plain sleep (0.889 ± 0.003).

5.3 Homeostasis and DFA: faster learning with backprop-free encoder credit assignment, not reliable learning

[CONFIRMATORY] With backprop, homeostasis raised the AUC by $+0.200$ [$+0.024$, $+0.377$] (raw $p = 0.029$), which does not survive Holm correction ($p = 0.117$); its solve rate was 10/10 vs 8/10 ($p = 0.474$).

[CONFIRMATORY] With backprop-free encoder credit assignment by DFA (heads use exact gradients; prospectively specified, 10 seeds per arm, seeds 1–10 in both arms), homeostasis raised the solve rate from 2/10 to 7/10 (difference $+0.50$ [$+0.07$, $+0.74$]; Fisher $p = 0.070$, Holm 0.070, not significant) and the AUC from 0.076 ± 0.101 to 0.402 ± 0.307 (difference $+0.326$ [$+0.101$, $+0.551$], $5.3\times$; Welch $p = 0.009$, Holm 0.017, significant). The remaining homeostasis seeds still failed outright (Figure 2). [INTERP] Homeostasis is a real but partial fix for DFA: it speeds up learning without a weight-transport requirement in the encoder, but it does not make DFA reliable on this task, and backprop with homeostasis (10/10) remains better. [EXPLORATORY] In continual learning, a DFA+homeostasis encoder with sleep reached only 0.450 ± 0.026 ACC on fetch3.

Table 3: Final ACC (mean $\pm$ SD over seeds) and FORGET on the continual sequences (exploratory). “Sleep, matched” consumes exactly as many replayed states as the same-seed replay run (SNN+Transformer only; confirmatory test in Section 5.4). Seeds 1–3 in every arm, except the SNN+Transformer column (replay and sleep, matched: seeds 1–8; sleep: 1–6; naive and isolation: 1–3).

	CNN, 3 tasks	SNN, 3 tasks	SNN+Transformer, 3 tasks	CNN, 5 tasks
<i>Final ACC</i>				
naive	0.555 $\pm$ 0.013	0.550 $\pm$ 0.130	0.452 $\pm$ 0.016	0.416 $\pm$ 0.098
EWC ($\lambda=100$)	0.674 $\pm$ 0.030	0.716 $\pm$ 0.199	–	–
replay	0.970 $\pm$ 0.009	0.814 $\pm$ 0.162	0.862 $\pm$ 0.038	0.926 $\pm$ 0.049
sleep	0.978 $\pm$ 0.001	0.889 $\pm$ 0.003	0.887 $\pm$ 0.045	0.928 $\pm$ 0.065
sleep, matched	–	–	0.879 $\pm$ 0.043	–
isolation	0.979 $\pm$ 0.001	0.833 $\pm$ 0.073	0.832 $\pm$ 0.037	0.976 $\pm$ 0.000
<i>FORGET (mean)</i>				
naive	0.631	0.321	0.536	0.612
replay	0.002	0.023	0.019	0.006
sleep	0.001	0.006	0.006	0.003
sleep, matched	–	–	0.020	–
<i>sleep – replay, final ACC [95% Welch CI] (unmatched budget)</i>				
	+0.008 [–0.015, +0.030]	+0.074 [–0.328, +0.476]	+0.024 [–0.026, +0.075]	+0.002 [–0.133, +0.137]

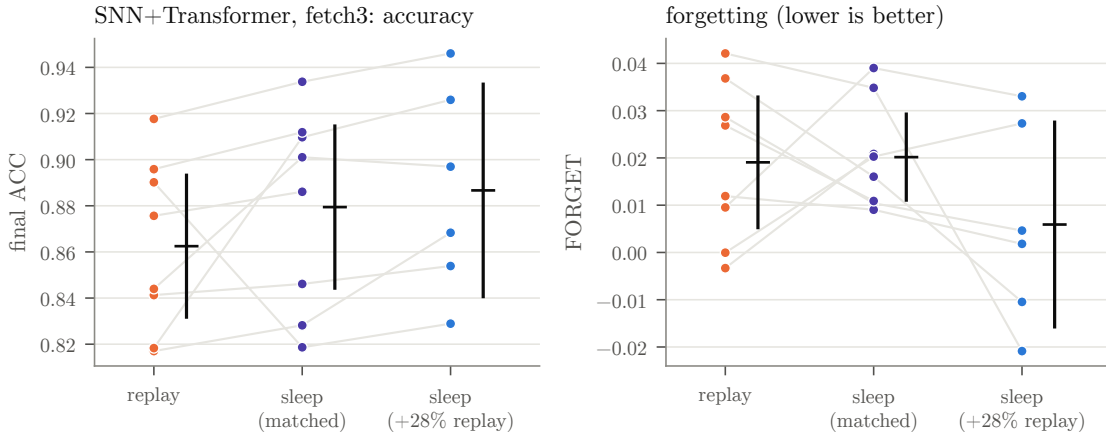


Figure 4: Sleep versus replay at an exactly matched replay budget (SNN+Transformer agent, fetch3; confirmatory for final ACC, prospectively specified; 8 seeds per arm, seeds 1–8 in both arms). *Replay*: CLEAR-style replay. *Sleep (matched)*: each run replays exactly as many stored states as the replay run with the same seed. *Sleep (+28%)*: the original sleep runs (seeds 1–6), which replayed 28% more states. Grey lines connect runs with the same seed; black bars are means with 95% CIs. Left: final accuracy (ACC). Right: forgetting (FORGET; lower is better; descriptive).

5.4 Sleep-like consolidation: no detectable difference from replay at a matched replay budget

[EXPLORATORY] Sleep and replay both remove most of naive fine-tuning’s forgetting on every agent (Figure 3, Table 3); EWC protects old tasks at a plasticity cost that grows with λ (return on the last task right after training it: 0.603, 0.441 and 0.280 for $\lambda = 10, 100, 1000$, against 0.971 for naive; CNN). At unmatched replay budgets, sleep never differs significantly from replay (last row of Table 3).

[CONFIRMATORY] **Replay budget matched.** On the SNN+Transformer agent, where the original sleep runs had consumed 28% more replayed states than replay, sleep with an exactly matched budget was not distinguishable from replay on final accuracy (0.879 ± 0.043 vs 0.862 ± 0.038 ; difference $+0.017$ [–0.026, +0.060], $p = 0.415$, 8 seeds per arm, the same seeds in both arms). Unlike the STDP contrasts, this interval is narrow. [DESCRIPTIVE] On forgetting, specified as descriptive in the log, the arms were also close (0.020

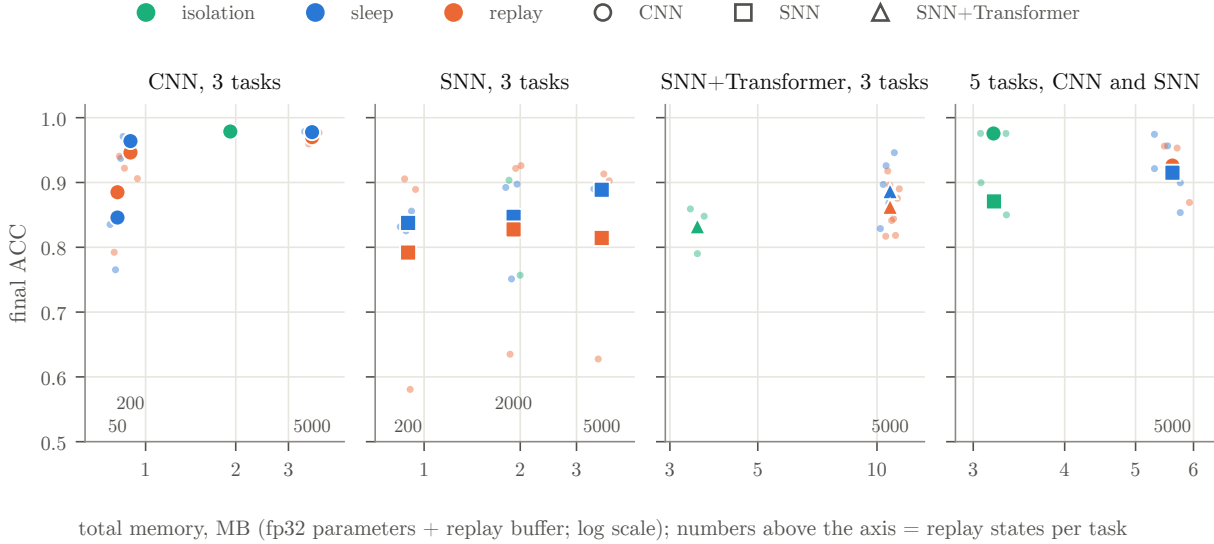


Figure 5: Final accuracy (ACC) against total memory (descriptive; the SNN comparisons with isolation are confirmatory, Section 5.5): fp32 parameters plus the replay buffer at its measured storage cost, log scale. Small dots: seeds; large markers: means. Numbers above the x -axis: replay states stored per task by sleep and replay; isolation stores none but trains one full-width network per task. At the default 5,000 states per task, isolation needs the least memory; with 200 states per task, sleep reaches a similar mean accuracy in less memory (3 seeds; wide CI). The 5-task SNN runs use the fair budget of 450k frames per task (Section 6). The post-freeze quarter-width isolation result (Section 5.5) is not plotted.

± 0.011 vs 0.019 ± 0.017 ; difference $+0.001 [-0.015, +0.017]$, $p = 0.881$) (Figure 4). On the same seeds, the unmatched sleep runs forgot 0.006 on average and the matched ones 0.022; matched minus unmatched ACC was $-0.014 [-0.029, +0.002]$ (paired, $p = 0.069$). [INTERP] Sleep’s small earlier edge on this agent came from its extra replay, not from consolidating offline. This matches the literature point that replay is a very strong baseline (Rolnick et al., 2019): in our implementation sleep is a way of scheduling replay, and at an equal replay budget we found no advantage for it. This is one sleep schedule, on one agent and sequence, at the full replay budget; it does not show that offline consolidation cannot help elsewhere.

5.5 Shared trunk with per-task heads versus isolation: memory-efficient only with small buffers and full-width isolation

[DESCRIPTIVE] Per parameter, the shared trunk with sleep is clearly more efficient than isolation: ACC per 100k parameters 0.50 vs 0.20 (CNN), 0.46 vs 0.17 (SNN), 0.27 vs 0.09 (SNN+Transformer), 0.41 vs 0.12 (CNN, 5 tasks). But at the default 5,000 stored states per task the buffer outweighs the extra networks: isolation uses less total memory and gets more accuracy per MB everywhere (0.51 vs 0.27, 0.43 vs 0.25, 0.24 vs 0.08, 0.31 vs 0.17 per MB; Figure 5).

[DESCRIPTIVE] With small buffers the shared trunk can be the more memory-efficient one, against full-width isolation. On the CNN, sleep with 200 states per task reaches 0.964 ± 0.009 in 0.89 MB (1.08 per MB, against 0.51 for isolation). [CONFIRMATORY] On the SNN, sleep with 200 states per task reaches a similar mean accuracy to isolation (0.838 ± 0.016 vs 0.833 ± 0.073 ; difference $+0.005 [-0.167, +0.177]$, $p = 0.921$, 3 seeds per arm, the same seeds in both arms) in 0.89 MB against 1.92 MB, 46% of the memory. With 3 seeds the interval is wide: the data rule out neither a large deficit nor a large advantage, so this is not evidence of equal accuracy. At *equal* total memory (SNN, 2000 states per task, 1.91 vs 1.92 MB) sleep does not beat isolation (difference $+0.014 [-0.165, +0.193]$, $p = 0.836$).

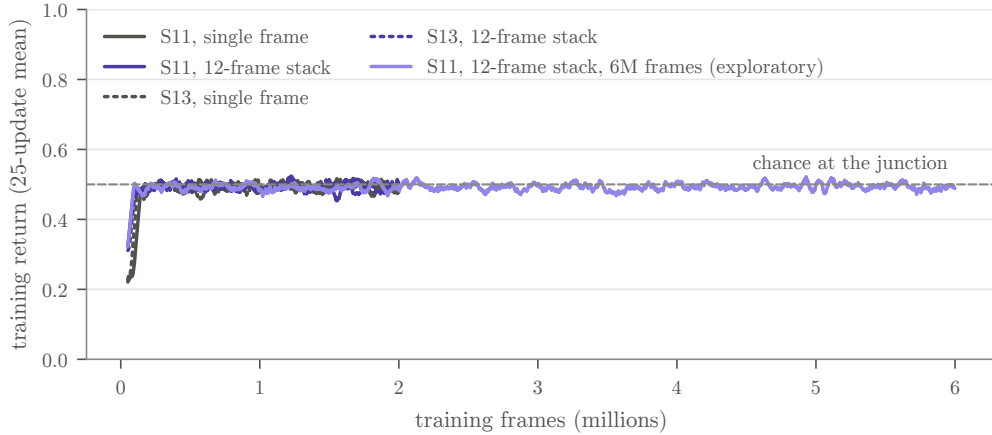


Figure 6: Memory-benchmark probes (sizing runs whose decision rule was prospectively specified; CNN encoder, one seed each, learning rate 0.0003). Training return (moving mean over 25 updates) on MiniGrid-MemoryS11 and S13 with a single frame (grey) or a stack of the last 12 frames (violet); light violet: an exploratory 6M-frame run of the stacked CNN on S11. A return of 0.5 is chance at the junction. No run learns to use the cue, so neither map tests working memory.

[EXPLORATORY] Forward transfer depends on the encoder: positive where learning from scratch is slow (SNN 0.199 ± 0.079 , $p = 0.049$; SNN+Transformer 0.423 ± 0.244 , $p = 0.095$), near zero on the 3-task CNN (-0.017 ± 0.141) and negative on the 5-task CNN (-0.525 ± 0.722). [CONFIRMATORY] On the 5-task SNN at the fair budget it was 0.306 ± 0.170 ($p = 0.089$).

Post-freeze follow-up: isolation’s network width. [POST-FREEZE] [CONFIRMATORY] The comparisons above use isolated networks of the same width as the shared trunk (159,864 parameters each). After the main results were frozen, we tested whether narrower isolated networks change the memory comparison, in a follow-up whose hypotheses, test, seed count and stopping rule were prospectively specified in its experiment log (`explore/EXPLORE.md`, comparison C4) before its runs. On the 5-task CNN sequence, isolation with quarter-width networks (convolution channels 8, 16, 32, 64 features, 64 hidden units; 44,928 parameters per task, 224,640 in total, 0.90 MB) reached a final ACC of 0.972 ± 0.002 . A shared trunk with per-task heads trained with LwF and an int8 teacher stores only its parameters (228,632, 0.91 MB, plus a transient 0.23 MB teacher during training); it reached 0.944 ± 0.018 (difference $+0.028$ [$+0.015$, $+0.041$], Holm $p = 0.002$). The same trunk with replay (973 stored states per task, 1.83 MB) reached 0.920 ± 0.052 (difference $+0.052$ [$+0.015$, $+0.089$], Holm $p = 0.011$). Full-width isolation (3.20 MB) reached 0.975 ± 0.001 ; quarter width minus full width was -0.004 [-0.005 , -0.002] (Holm $p = 0.002$), inside the prospectively specified ± 0.05 margin for “no large difference”. All four arms used seeds 201–210 (10 per arm) and the frozen CNN protocol (150k frames per task, learning rate 0.001, 100 evaluation episodes per task); LwF and replay processed the same number of distilled or replayed states (1.19M per run). [INTERP] On this sequence the frozen isolation baseline’s per-task networks were much larger than the tasks need, so “a shared trunk is more memory-efficient than isolation with small buffers” holds only against full-width isolation. Two limits apply. The shared trunk was not shrunk in the same way, so this is not a width-matched comparison of the two strategies, and the follow-up used LwF and replay rather than sleep. The SNN case is untested: the only narrow SNN isolation runs were a two-seed pilot at 150k frames per task, a budget at which even full-width fresh SNNs often had not learned their tasks (Section 6).

[INTERP] The shared trunk’s advantage in the frozen study is memory efficiency when replay is cheap and isolation is full width, not higher accuracy at equal memory. “Shared weights are more efficient than isolation” is only true with the buffer size and the isolation width stated.

5.6 Transformer working memory: inconclusive, because benchmark validity was not established

[EXPLORATORY] On the fetch tasks, which are fully determined by the current frame, the Transformer has nothing to remember; the SNN+Transformer agent needed 400k frames per task where the CNN and SNN agents used 150k. To test memory we used MiniGrid-Memory. On S7 (pilot, one seed, 1M frames) a memoryless CNN already succeeded in 0.915 of episodes (a 4-frame stack: 0.995), so S7 does not isolate memory: in that small map the cue and both choices are visible from one position. The pilot’s SNN runs (SNN with an 8-frame stack 0.465, SNN+Transformer 0.970) used different learning rates (0.001 vs 0.0003) and cannot support a claim.

[CONFIRMATORY] For the prospectively specified benchmark we probed S11 and S13, where the cue is further from the junction than the view reaches. The memoryless CNN was at chance, as required (0.430 and 0.485 success), but so was a CNN with a 12-frame stack (0.465 and 0.460), with episodes of about 10–12 steps (Figure 6). Neither map met the prospectively specified criterion, so, by the rule written before the probes, the Transformer comparison was not run. [EXPLORATORY] A run added after the probes, of the stacked CNN on S11 for 6M frames, stayed at chance (0.485; training return 0.48–0.51 throughout). [INTERP] The episode lengths show that no agent learned to go back and look at the cue; they walk to the junction and guess. The obstacle is exploration and credit assignment, not context length. The question whether Transformer working memory helps is therefore inconclusive, not answered in either direction (Section 7).

6 What did not survive controls

Several results looked positive before the controls were in place. Table 4 lists them with the control that removed them. All “before” numbers are recomputed from the same frozen runs, restricted to the seeds that existed at the time; all of them are exploratory. The last row comes from the post-freeze follow-up.

Seed count. [INTERP] A handful of seeds was enough to produce $p = 0.072$ (sleep vs replay) and $p = 0.088$ (homeostasis vs backprop), and a 3/3 vs 1/5 split that looked decisive (DFA). All three shrank once the seed count was raised. With bimodal outcomes, one failed seed moves a small-sample mean a lot.

Training budget. [INTERP] At 150k frames per task, a freshly initialised SNN often had not learned its task yet, so isolation (which starts from scratch on every task) was under-trained while the shared trunk benefited from transfer. At a budget where fresh networks learn (450k, chosen by single-task probes before the runs), the gap shrank from +0.303 [+0.148, +0.457] to +0.044 [−0.009, +0.098]. Sleep still learned more tasks to 0.8 or above (14/15 vs 12/15), which is consistent with some positive transfer.

Memory accounting. [INTERP] Comparing per parameter ignores the replay buffer, which is the price the shared-trunk methods pay for not forgetting. With the buffer counted, the conclusion reversed at the default buffer size.

Network width. [INTERP] Comparing per unit of memory still fixed isolation’s width at the shared trunk’s. With quarter-width isolated networks, isolation beat both shared-trunk methods at equal or lower memory on the 5-task CNN sequence, and differed from full width by only −0.004 [−0.005, −0.002]. The small-buffer exception therefore depends on a width choice that the frozen study did not vary.

Replay-sample budget. [INTERP] Sleep and replay are usually compared at equal buffer size, but the number of replayed samples they consume can differ a lot (here by 28% on average). Matching it removed the difference.

Benchmark validity. [INTERP] A memory benchmark must fail without memory and be learnable with it. S7 failed the first condition; S11 and S13 failed the second at our budget.

Table 4: Earlier positive findings that shrank or disappeared once a control was added. *Evidence then:* the comparison as it stood, recomputed from the frozen runs restricted to the seeds available at the time (all exploratory). *After the control:* the comparison with the control in place; for the last row, the post-freeze follow-up, which used the 5-task CNN sequence with LwF and replay as the shared-trunk methods (Section 5.5). *Label:* evidential status of the “after” result (conf. = confirmatory, desc. = descriptive). *p:* Welch unless noted. All arms of each comparison share their seed set (Section 3.5).

Earlier finding	Evidence then	After the control	Cause	Label
Sleep beats replay (SNN+Transformer)	seeds 1–3: +0.039 [−0.006, +0.084], $p = 0.072$	seeds 1–6: $p = 0.224$; budget-matched, 8 seeds: +0.017 [−0.026, +0.060], $p = 0.415$	seed count; replay-sample budget	conf.
Shared trunk far more efficient than isolation	ACC per parameter: 2.5–3.3×	per total memory at the default buffer, isolation is 1.8–2.9× more efficient; with 200 states/task (SNN), similar mean ACC (0.838 vs 0.833; 3 seeds, wide CI) in less than half the memory	memory accounting	desc.; conf. (SNN)
Sleep far beats isolation on 5 tasks (SNN)	150k/task, 2 seeds: 0.778 ± 0.007 vs 0.475 ± 0.023, $p = 0.023$	450k/task, 3 seeds: 0.915 ± 0.014 vs 0.871 ± 0.026, +0.044 [−0.009, +0.098], $p = 0.078$	training budget	conf.
Homeostasis helps backprop	3 vs 5 seeds (homeostasis vs backprop): 3/3 solved, $p = 0.088$	10 seeds: +0.200 [+0.024, +0.377], $p = 0.029$, Holm 0.117	seed count; multiple comparisons	conf.
Homeostasis makes DFA work	3/3 vs 1/5 solved, AUC $p < 0.001$	10 seeds: 7/10 vs 2/10, Fisher $p = 0.070$ (AUC effect survives)	seed count	conf.
Transformer helps on a memory task	S7 pilot: 0.970 vs 0.465 (frame stack)	memoryless CNN on S7: 0.915; on S11/S13 no memory agent learns	benchmark validity; learning-rate confound	conf. (rule)
With a small buffer, a shared trunk is more memory-efficient than isolation	against full-width isolation: CNN, sleep with 200 states/task 1.08 vs 0.51 ACC per MB; SNN as in row 2	quarter-width isolation, CNN 5 tasks, 10 seeds: 0.972 at 0.90 MB vs shared trunk with LwF 0.944 at 0.91 MB (+0.028 [+0.015, +0.041]) and with replay 0.920 at 1.83 MB (+0.052 [+0.015, +0.089])	network width not matched	conf., post-freeze

7 Limitations

Task identity is given. Our setting is task-incremental: the agent knows which task it is doing. Removing the task identity from fetch3 would make it unsolvable (the goal is not visible), so a task-agnostic study needs a different benchmark, and sleep’s distillation, which relies on per-task heads, would need redesigning.

Scale. All agents are small (159,864–327,800 parameters), the environments are MiniGrid gridworlds, sequences have 3 or 5 tasks, and budgets are 150k–450k frames per task. Results may not transfer to larger agents, longer sequences or visually rich tasks.

CPU only, few seeds. Everything ran on a 4-core CPU. Many continual arms have only 3 seeds, so their CIs are wide and a tie means at most “no large difference” where the interval is narrow, never equivalence. The prospectively specified comparisons used 8–10 seeds per arm, and the post-freeze follow-up 10.

Energy is an estimated arithmetic energy proxy. The energy numbers multiply operation counts by literature constants for 45nm arithmetic and assume event-driven hardware. We measured no power. The only cost we measured is CPU time in our implementation, and there the SNN was slower.

Single settings. Hyperparameters (homeostasis target, α , EWC λ beyond three values, sleep schedule) were not tuned per arm. A different homeostasis target or sleep schedule could change the results.

Baselines are simple. In the frozen study, isolation uses one full-width network per task; the post-freeze follow-up shows that narrower isolated networks change the memory comparison (Section 5.5), and PackNet-style masks would be cheaper still. The shared trunk was never shrunk in the same way. Our sleep phase is replay plus distillation, not the sleep replay consolidation of Tadros et al. (2022).

The Transformer question is inconclusive. On S7 memory was unnecessary; on S11 and S13 no agent, including a CNN with a 12-frame stack, learned to use the cue, even at 6M frames. Benchmark validity was therefore not established, and the comparison was not run.

Prospective specification was informal. The test plans were recorded in the project’s version-controlled experiment log before the runs, not in a public registry. The first two phases were exploratory, and their positive findings are exactly the ones Section 6 revisits. The follow-up was planned after the frozen results were known, although each of its tests was specified before its own runs.

8 Future work

These are plans and open questions, not results. The main limitation of this study is scale: every experiment ran on a 4-core CPU, where CPU-bound environment stepping, not model size, was the bottleneck. A natural next step is to port the agent, the mechanisms and the baselines to JAX and run them on GPUs with XLand-MiniGrid (Nikulin et al., 2024), a GPU-native implementation of MiniGrid with a rule-and-goal system for generating many related tasks, after first replicating the headline results on its MiniGrid ports (sleep versus replay at a matched replay budget, STDP versus a same-size random update, and the isolation-width result). Open questions for such a study are: longer sequences of 10–50 related tasks where transfer matters, with full accuracy-versus-memory curves in which every method shrinks along its own size setting (isolation width, shared-trunk width, buffer size); 10–20 seeds per arm, with a power analysis and equivalence tests, so that “no large effect” nulls become informative; a memory benchmark that agents can learn, followed by Transformer versus GRU versus frame stack and a stateful SNN; the SNN version of the isolation-width comparison at an adequate training budget (450k frames per task); whether the conclusions change with network size; and a task-agnostic setting with observable goals. Smaller follow-ups are a replay-budget sweep for sleep versus interleaved replay (motivated by an exploratory observation in the follow-up log that sleep forgot more when replay was scarce), separating offline replay from self-distillation within sleep, plots of the STDP runaway mechanism, a stronger CNN sparsifier such as k-winners-take-all, compressed or generative replay (Shin et al., 2017) to shrink the buffer, a PackNet variant that can regrow capacity, and one standard benchmark such as Continual World (Wołczyk et al., 2021) for comparability. The methodology would stay the same: prospectively specified tests, fresh seeds for confirmation, matched resources, and a log of every failure.

9 Conclusion

We isolated six brain-inspired mechanisms in a small task-incremental continual RL setting and gave each a matched control; a seventh, Transformer working memory, remained inconclusive because benchmark validity was not established. [OURS] No mechanism produced a statistically reliable improvement in final accuracy or forgetting beyond its matched control; given the seed counts, these results rule out large effects rather than show equivalence, and only where the intervals are narrow. *Survived*: homeostasis sped up learning of a spiking encoder with backprop-free encoder credit assignment (DFA; heads use exact gradients), without making it reliable. *Conditional*: with a small replay buffer, a shared trunk with per-task heads and sleep reached similar mean accuracy to full-width isolation in less than half the memory, but in a post-freeze follow-up quarter-width isolation beat shared-trunk methods at equal or lower memory on a 5-task CNN sequence; and the spiking encoder reached a lower operation-count regime whose energy value is an estimated arithmetic proxy and whose measured CPU cost in our implementation was higher. *Did not survive*: the tested stabilised STDP rule against a same-size random update, homeostasis with backprop, and sleep against replay at a matched replay budget, together with the earlier positive findings of Section 6. *Unresolved*: the Transformer question and the SNN version of the isolation-width comparison. [INTERP] The most consistent

lesson is methodological: in this setting, the earlier positive effects of biological mechanisms shrank or disappeared once seed counts, training budgets, replay budgets, memory accounting and, after the freeze, network width were made fair; the speed-up that homeostasis gives DFA survived. We suggest that claims of this kind are reported with those budgets stated and matched.

Reproducibility statement

All code, configuration files, the frozen run directories and the analysis scripts are in an anonymised repository provided as supplementary material. For each of the 289 PPO runs the repository keeps the full configuration, the training log, the evaluation results and the final checkpoint, so every number can be traced to its run and every run repeated with its seed. There are three levels of reproduction.

1. *The paper from the frozen results* (minutes, no training). `python paper/compute_numbers.py` regenerates every number in the text; `paper/NUMBERS.md` lists each one with its source file. `python paper/figures/make_figures.py` regenerates the figures, and `latexmk -pdf main.tex` builds the paper. The statistical tests are rerun by `scripts/rq1_stats.py`, `scripts/rq5_stats.py`, `scripts/s4_stats.py` and `scripts/memory_fair.py`; the post-freeze follow-up's tests by `explore/confirm_stats.py`, and its experiment log, with the prospective specification and every pilot, is `explore/EXPLORE.md`.
2. *A single run.* `python train.py --config configs/<name>.yaml --set seed=<s>` for a single task and `python continual.py --suite fetch3 --method <method> --config configs/<name>.yaml --seed <s>` for a task sequence; any stored configuration key can be overridden with `--set`. The follow-up's runs use `python explore/run_cl.py` with the same flags (the job lists are in `explore/jobs/`).
3. *All experiments.* The job lists in `jobs/` run through `scripts/run_queue.py` (resumable, one thread per job). The frozen runs took about 110 single-thread CPU hours in total, on a 4-core machine without a GPU.

The software environment is Python with PyTorch (CPU), Gymnasium and MiniGrid (`requirements.txt`); the unit tests run with `pytest tests/`. The `demo/` folder holds an offline results page, GIFs of the naive and sleep agents after the full fetch3 sequence (final accuracy-matrix rows 0.126, 0.542, 0.956 and 0.978, 0.979, 0.979), and a script that runs a trained agent for one episode on CPU.

AI assistance disclosure

TODO: Author to write: which parts of the code, experiments, analysis and text were produced with AI assistance, and how they were checked.

References

- Guo-qiang Bi and Mu-ming Poo. Synaptic modifications in cultured hippocampal neurons: Dependence on spike timing, synaptic strength, and postsynaptic cell type. *Journal of Neuroscience*, 18(24):10464–10472, 1998.
- Arslan Chaudhry, Puneet K. Dokania, Thalaiyasingam Ajanthan, and Philip H. S. Torr. Riemannian walk for incremental learning: Understanding forgetting and intransigence. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.
- Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo de Lazcano, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023.

- Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bannamoun, Doo Seok Jeong, and Wei D. Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9):1016–1054, 2023.
- Wei Fang, Yanqi Chen, Jianhao Ding, Zhaofei Yu, Timothée Masquelier, Ding Chen, Liwei Huang, Huihui Zhou, Guoqi Li, and Yonghong Tian. SpikingJelly: An open-source machine learning infrastructure platform for spike-based intelligence. *Science Advances*, 9:eadi1480, 2023.
- Nicolas Frémaux and Wulfram Gerstner. Neuromodulated spike-timing-dependent plasticity, and theory of three-factor learning rules. *Frontiers in Neural Circuits*, 9:85, 2016.
- Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- Mark Horowitz. Computing’s energy problem (and what we can do about it). In *IEEE International Solid-State Circuits Conference (ISSCC)*, pp. 10–14, 2014.
- Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and João G. M. Araújo. CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022.
- Bingqiang Huo, Fang Li, Siyu Peng, Hongwei Chen, Sudan Xin, and Hongjun Wang. Research on SNN learning algorithms and networks based on biological plausibility. *IEEE Access*, 2025. doi: 10.1109/ACCESS.2025.3566717.
- Khimya Khetarpal, Matthew Riemer, Irina Rish, and Doina Precup. Towards continual reinforcement learning: A review and perspectives. *Journal of Artificial Intelligence Research*, 75:1401–1476, 2022.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharmashan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Choongseop Lee, Yuntae Park, Sungmin Yoon, Jiwoon Lee, Youngho Cho, and Cheolsoo Park. Brain-inspired learning rules for spiking neural network-based control: A tutorial. *Biomedical Engineering Letters*, 15: 37–55, 2025. doi: 10.1007/s13534-024-00436-6.
- Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, 2018.
- Arun Mallya and Svetlana Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019.
- Robert G. Newcombe. Interval estimation for the difference between independent proportions: Comparison of eleven methods. *Statistics in Medicine*, 17(8):873–890, 1998.
- Alexander Nikulin, Vladislav Kurenkov, Ilya Zisman, Artem Agarkov, Viacheslav Sinii, and Sergey Kolesnikov. XLand-MiniGrid: Scalable meta-reinforcement learning environments in JAX. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.
- Arild Nøkland. Direct feedback alignment provides learning in deep neural networks. In *Advances in Neural Information Processing Systems*, 2016.
- Vishal Pandey and Debasmita Biswas. Spiking decision transformers: Local plasticity, phase-coding, and dendritic routing for low-power sequence control. arXiv:2508.21505, 2025.

- David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy P. Lillicrap, and Greg Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, 2019.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.
- Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems*, 2017.
- Timothy Tadros, Giri P. Krishnan, Ramyaa Ramyaa, and Maxim Bazhenov. Sleep-like unsupervised replay reduces catastrophic forgetting in artificial neural networks. *Nature Communications*, 13:7742, 2022. doi: 10.1038/s41467-022-34938-7.
- Gina G. Turrigiano and Sacha B. Nelson. Homeostatic plasticity in the developing nervous system. *Nature Reviews Neuroscience*, 5(2):97–107, 2004.
- Korneel Van den Berghe, Stein Stroobants, Vijay Janapa Reddi, and G. C. H. E. de Croon. Adaptive surrogate gradients for sequential reinforcement learning in spiking neural networks. In *Advances in Neural Information Processing Systems*, 2025.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- B. L. Welch. The generalization of “Student’s” problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35, 1947.
- Maciej Wołczyk, Michał Zając, Razvan Pascanu, Łukasz Kuciński, and Piotr Miłoś. Continual world: A robotic benchmark for continual reinforcement learning. In *Advances in Neural Information Processing Systems*, 2021.
- Hongzhuo Yu, Fei Zhu, Guo-Sen Xie, and Ling Shao. Self-consolidation for self-evolving agents, 2025. TODO: venue and year not printed on the local PDF; check before submission.
- Chenlin Zhou, Liutao Yu, Zhaokun Zhou, Han Zhang, Jiaqi Wang, Huihui Zhou, Zhengyu Ma, and Yonghong Tian. Spikingformer: A key foundation model for spiking neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

A Protocol constants

PPO	16 envs $\times$ 128 steps, 4 epochs, minibatch 256, clip 0.2, $\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.95$, KL stop 0.03
Learning rate	0.001 (CNN, SNN), 0.0003 (SNN+Transformer, memory probes)
Frames per task	150k (CNN, SNN), 400k (SNN+Transformer), 450k (5-task SNN, fair)
SNN	4 timesteps, learnable leak (init 0.9), threshold 1, surrogate slope 25, membrane reset per frame
Plasticity	$\alpha = 0.03$ (prospectively specified), homeostasis target 0.15, rate 0.01
Sleep	every 25 updates + end of task, 700 steps per phase, 5,000 states per task
Shared trunk (CNN)	trunk 142,832 parameters; per task one head (17,032) and one task embedding (128)
Post-freeze C4	CNN, 5 tasks; quarter-width isolation: channels 8, 16, 32, 64 features, 64 hidden units; LwF: 128 distilled states per PPO minibatch, int8 teacher; replay: 973 states per task
Evaluation	100 episodes per task (continual), 200 (single task), sampled actions
Memory accounting	188 B/state (CNN, SNN), 632 B/state (SNN+Transformer)

B Additional numbers

EWC on the CNN (3 tasks): ACC 0.816 ± 0.120 , 0.674 ± 0.030 and 0.545 ± 0.006 for $\lambda = 10, 100, 1000$ (forgetting $-0.005, -0.002, 0.000$). Isolation on 5 tasks at the fair budget: 3.20 MB, ACC 0.871; sleep 5.62 MB, ACC 0.915. SNN replay with small buffers: 200 states per task 0.792 ± 0.183 , 2000 states per task 0.828 ± 0.167 . The complete list of numbers, with sources, is `paper/NUMBERS.md`.